

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	<i>Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.</i>		
Student Name:	K.KAVIYA	Reg. No.:	192521262
Section / Batch:	1	Date:	17/07/2026

Code of Conduct

I _____ k.kaviya _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: _____ k.kaviya _____

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:		100 Marks	

Analytical Problem Solving

Question 1: Linked List for Weekly Product Sales

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

struct Node
{
    int productID;
    char day[15];
    int unitsSold;
    struct Node *next;
};

struct Node *head = NULL;

// Insert sales data
void insertSale(int id, char d[], int units)
{
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->productID = id;
    strcpy(newNode->day, d);
    newNode->unitsSold = units;
    newNode->next = NULL;
```

```
if(head == NULL)
{
    head = newNode;
}
else
{
    struct Node *temp = head;
    while(temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;
}
}
```

// Display sales of one product

```
void displayProductSales(int id)
```

```
{
    struct Node *temp = head;
    int found = 0;

    printf("\nSales Details of Product %d\n", id);

    while(temp != NULL)
    {
        if(temp->productID == id)
        {
            printf("%-10s : %d units\n", temp->day, temp->unitsSold);
        }
    }
}
```

```

        found = 1;
    }
    temp = temp->next;
}

if(found == 0)
    printf("Product not found.\n");
}

// Total sales for each product
void totalSales()
{
    int ids[20];
    int totals[20];
    int count = 0;
    int i;

    struct Node *temp = head;

    while(temp != NULL)
    {
        int found = 0;

        for(i=0;i<count;i++)
        {
            if(ids[i] == temp->productID)
            {
                totals[i] += temp->unitsSold;
            }
        }
    }
}

```

```
        found = 1;
        break;
    }
}
```

```
if(found == 0)
{
    ids[count] = temp->productID;
    totals[count] = temp->unitsSold;
    count++;
}
```

```
temp = temp->next;
}
```

```
printf("\nTotal Weekly Sales\n");
for(i=0;i<count;i++)
{
    printf("Product %d = %d units\n", ids[i], totals[i]);
}
}
```

// Highest sales

```
void highestSale()
{
    struct Node *temp = head;
    int max = -1;
    int pid;
```

```
char day[15];
```

```
while(temp != NULL)
```

```
{
```

```
    if(temp->unitsSold > max)
```

```
    {
```

```
        max = temp->unitsSold;
```

```
        pid = temp->productID;
```

```
        strcpy(day, temp->day);
```

```
    }
```

```
    temp = temp->next;
```

```
}
```

```
printf("\nHighest Sales\n");
```

```
printf("Product ID : %d\n", pid);
```

```
printf("Day      : %s\n", day);
```

```
printf("Units Sold : %d\n", max);
```

```
}
```

```
int main()
```

```
{
```

```
    // Product 101
```

```
    insertSale(101,"Monday",20);
```

```
    insertSale(101,"Tuesday",15);
```

```
    insertSale(101,"Wednesday",18);
```

```
    insertSale(101,"Thursday",22);
```

```
    insertSale(101,"Friday",25);
```

```
    insertSale(101,"Saturday",30);
```

```
insertSale(101,"Sunday",28);

// Product 102
insertSale(102,"Monday",10);
insertSale(102,"Tuesday",12);
insertSale(102,"Wednesday",14);
insertSale(102,"Thursday",16);
insertSale(102,"Friday",18);
insertSale(102,"Saturday",21);
insertSale(102,"Sunday",19);

// Product 103
insertSale(103,"Monday",8);
insertSale(103,"Tuesday",9);
insertSale(103,"Wednesday",12);
insertSale(103,"Thursday",15);
insertSale(103,"Friday",17);
insertSale(103,"Saturday",20);
insertSale(103,"Sunday",18);

totalSales();
highestSale();
displayProductSales(101);

return 0;
}
```

Output:

Total Weekly Sales

Product 101 = 158 units

Product 102 = 110 units

Product 103 = 99 units

Highest Sales

Product ID : 101

Day : Saturday

Units Sold : 30

Sales Details of Product 101

Monday : 20 units

Tuesday : 15 units

Wednesday : 18 units

Thursday : 22 units

Friday : 25 units

Saturday : 30 units

Sunday : 28 units

Question 2: Evaluate the Expression**Expression:**

$(8+2)*(6-4)/2$

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int stack[20];  
    int top = -1;  
  
    // (8 + 2)  
    stack[++top] = 8;  
    stack[++top] = 2;  
    int a = stack[top--];  
    int b = stack[top--];  
    stack[++top] = b + a;    // 10  
  
    // (6 - 4)  
    stack[++top] = 6;  
    stack[++top] = 4;  
    a = stack[top--];  
    b = stack[top--];  
    stack[++top] = b - a;    // 2  
  
    // Multiply  
    a = stack[top--];  
    b = stack[top--];  
    stack[++top] = b * a;    // 20  
  
    // Divide by 2  
    stack[++top] = 2;  
    a = stack[top--];  
    b = stack[top--];  
    stack[++top] = b / a;    // 10
```

```
printf("Expression : (8+2)*(6-4)/2\n");  
printf("Result = %d\n", stack[top]);  
  
return 0;  
}
```

Output:

Expression : (8+2)*(6-4)/2

Result = 10